INFORME DE ANÁLISIS DE CALIDAD DE CÓDIGO
SONARQUBE

Bryan Nicolas Chaves Arce
ID: 368668

Ingeniería de sistemas, Facultad de ingeniería, Corporación Universitaria Minuto De Dios
NRC 61831: Desarrollo de software seguro
Ing. Edwin Albeiro Ramos Villamil

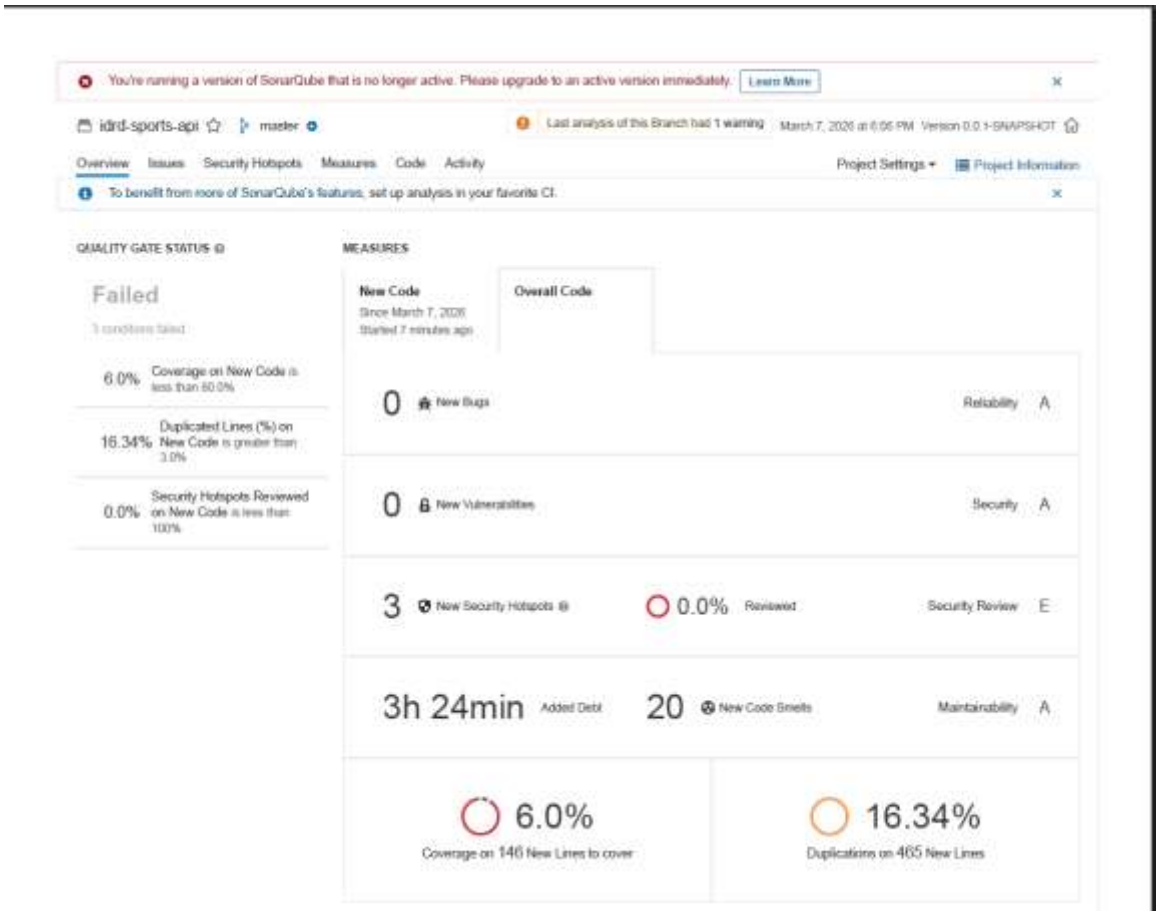
marzo 07, 2025

1. Introducción

Este documento presenta el análisis de calidad de código realizado sobre el proyecto 'idrd-sports-api' utilizando la herramienta SonarQube. El objetivo del análisis es identificar problemas de mantenibilidad, malas prácticas de programación, duplicación de código y debilidades en pruebas unitarias que puedan afectar la calidad del software.

2. Resumen del análisis

Resultados principales del análisis estático:



Métrica	Resultado
Bugs	0
Vulnerabilidades	0
Code Smells	20
Security Hotspots	3
Cobertura de código nuevo	6%
Duplicación de código	16.34%

3. Archivos con mayor número de problemas

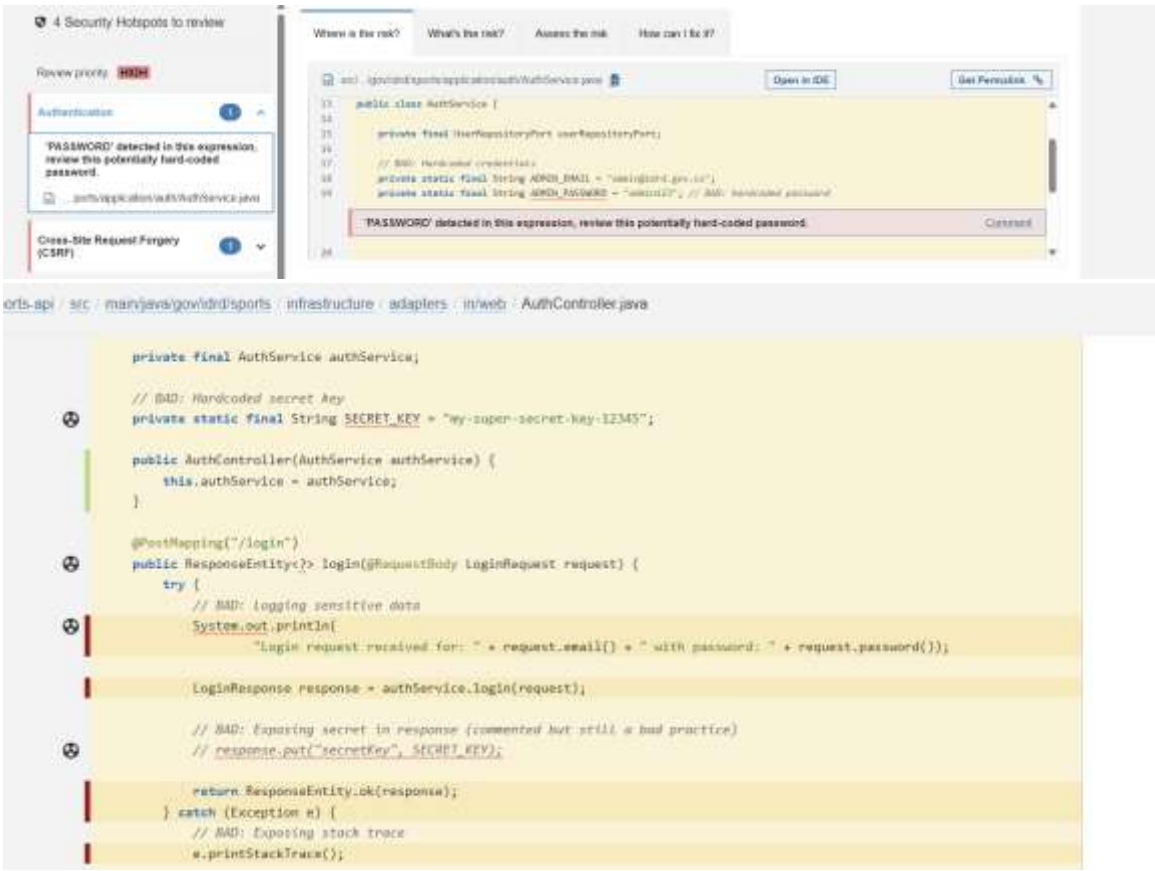
Los problemas detectados se concentran principalmente en los siguientes archivos:

- AuthController.java
- AuthService.java
- IdrdSportsApiApplicationTests.java

4. Principales problemas detectados

- Uso de System.out o System.err en lugar de un sistema de logging.
- Bloques de código comentado que deberían eliminarse.
- Uso de tipos genéricos wildcard (?) que afectan la legibilidad del código.
- Métodos y variables que no se utilizan.
- Uso de excepciones genéricas en lugar de excepciones específicas.
- Duplicación de literales en el código.
- Pruebas unitarias sin assertions.

Tokens o datos sensibles harcodeados.



Uso de System.out en vez de loggin

```java
            // BAD: Generic error handling
            Map<String, String> error = new HashMap<>();
            error.put("error", e.getMessage());
            error.put("details", e.toString()); // BAD: exposing internal details

            return ResponseEntity.status(HttpStatus.UNAUTHORIZED).body(error);
        }
    }

    @PostMapping("/register")
    public ResponseEntity<?> register(@RequestBody RegisterRequest request) {
        // BAD: No input validation
        // BAD: Logging password
        System.out.println("Registering user: " + request.email() + " password: " + request.password());

        try {
            authService.register(request);

            // BAD: Returning sensitive information
            Map<String, String> response = new HashMap<>();
            response.put("message", "User registered successfully");
            response.put("email", request.email());
            response.put("password", request.password()); // BAD: returning password

            return ResponseEntity.status(HttpStatus.CREATED).body(response);
        } catch (Exception e) {
```

Código no usado o comentariado

```java
            error.put("error", "Registration failed");

            return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(error);
        }
    }

    // BAD: Unused private method
    private String generateToken() {
        return "token-" + System.currentTimeMillis();
    }

    // BAD: Dead code
    /*
     * private void oldMethod() {
     * String x = "old";
     * }
     */
}
```

Imprimir errores al usuario que no son adecuados

5. Recomendaciones de mejora

- Reemplazar System.out por un framework de logging como SLF4J o Logback.
- Eliminar código muerto (métodos, variables y comentarios innecesarios).
- Crear excepciones personalizadas para manejar errores de forma adecuada.
- Reducir duplicación de código utilizando constantes y refactorización.
- Incrementar la cobertura de pruebas unitarias con JUnit y Mockito.
- Revisar los Security Hotspots identificados por SonarQube.

6. Conclusión

El análisis realizado con SonarQube evidencia que el proyecto no presenta errores críticos de seguridad, pero sí varios problemas de mantenibilidad y calidad del código. La mayoría de los problemas corresponden a malas prácticas de programación, duplicación de código y baja cobertura de pruebas. La corrección de estos problemas permitirá mejorar la calidad general del software y cumplir con los criterios del Quality Gate definidos en SonarQube.